

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: Mohammed Almas

Roll No.: A24126552098

Date: 25-08-2026

1. TITLE

Introduction to Node.js, Built-in Modules, Custom Modules and Express Server

2. AIM

To understand and implement basic Node.js programs using built-in modules, a custom module, and the Express framework, including file handling, operating system information, path handling, HTTP server creation, URL parsing and events.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the basic execution of a Node.js program.
 - To create and execute a simple Node.js application.
 - To use the fs module for creating and writing files.
 - To use the os module to obtain operating system and CPU information.
 - To use the path module to extract a file name from a path.
 - To create a basic HTTP server using the http module.
 - To parse URL components using the URL class from the url module.
 - To create and emit custom events using EventEmitter.
 - To understand how a custom module can be imported using require().
 - To create a simple Express application with a GET route and port 3000.
-

4. THEORY

Node.js is a JavaScript runtime environment used to execute JavaScript outside a web browser. It provides built-in modules that expose functionality for files, operating-system information, paths, networking, URLs and events. Node.js programs can also be organized into separate modules and can use external frameworks such as Express.

fs module: The fs module provides file-system operations. In the given program, `writeFileSync()` creates or overwrites `hello.txt` with the text `'hello node.js'`.

os module: The os module provides information about the operating system. The program prints the platform and the number of CPU cores.

path module: The path module provides utilities for working with file and directory paths. `basename()` extracts the final file name from a path.

http module: The http module can create a server. The given server responds with `'Hello from Node.js!'` and listens on port 3000.

url module: The URL class parses a URL into components such as protocol, host, pathname and query string.

events module: EventEmitter is used to create an event emitter, register a listener for the `'welcome'` event and emit that event.

Custom module: The `user.js` program imports a local module named `app` using `require('./app')` and calls `add()`, `sub()`, `mul()` and `div()` functions.

Express: Express is used to create a web application. The program defines a GET route for `'/'` and starts the application on port 3000.

5. ALGORITHM / PROCEDURE

1. Install Node.js and verify that Node.js can execute JavaScript files.
 2. Create a Node.js source file for the basic module demonstrations.
 3. Import the fs module and create hello.txt using writeFileSync().
 4. Import the os module and display the operating-system platform and CPU core count.
 5. Import the path module and display the basename of the specified file path.
 6. Import the http module and create a server that sends 'Hello from Node.js!'.
 7. Start the HTTP server on port 3000.
 8. Import the URL class and create a URL object for the given address.
 9. Display the protocol, host, pathname and query string.
 10. Import EventEmitter, register a listener for the welcome event and emit the event.
 11. Create or use the local app module and import it using require('./app').
 12. Call the add, sub, mul and div functions with 10 and 5.
 13. Create an Express application using express().
 14. Define a GET route for '/' that sends 'Hello from Express!'.
 15. Start the Express application on port 3000.
 16. Execute the programs and verify the displayed results and server responses.
-

6. PROGRAM

Hello.txt

```
Hello node.js
```

User.js

```
const math = require("./app");  
console.log("Add:", math.add(10,5));  
console.log("Sub:", math.sub(10,5));  
console.log("Mul", math.mul(10,5));  
console.log("Div", math.div(10,5));
```

module.js

```
const express = require("express");  
const app = express();  
app.get("/", (req, res) => {  
  res.send("Hello from Express!");  
});  
app.listen(3000, () => {  
  console.log("Server running on port 3000");  
});
```

builtin.js

```
//fs module
const fs = require("fs");
fs.writeFileSync("hello.txt","hello node.js");
console.log("File created");

//os module
const os = require("os");
console.log(os.platform()); //os which we are using
console.log(os.cpus().length); //no. of cpu cores

//path module
const path = require("path");
console.log(path.basename("C:/Users/anits/Desktop/A24126552126/week5/hello.txt"));

// http module
const http = require("http");
const server = http.createServer((req, res) => {
  res.end("Hello from Node.js!");
});
server.listen(3000);
console.log("Server running on port 3000");

// url module
const { URL } = require("url");
const myURL = new URL("https://example.com:8080/products?id=101");
console.log("Protocol:", myURL.protocol);
console.log("Host:", myURL.host);
console.log("Path:", myURL.pathname);
console.log("Query:", myURL.search);

//events module
const EventEmitter = require("events");
const myEvent = new EventEmitter();
myEvent.on("welcome", () => {
  console.log("Welcome to Node.js!");
});
myEvent.emit("welcome");
```

7. CODE EXPLANATION

Code	Explanation
const fs = require('fs'	Imports the built-in file-system module.
fs.writeFileSync()	Writes 'hello node.js' into hello.txt synchronously.
const os = require('os'	Imports the operating-system module.
os.platform()	Returns the platform/operating system identifier.
os.cpus().length	Returns the number of CPU cores reported by Node.js.
path.basename()	Returns the last portion of the specified path, which is the file name.
http.createServer.	Creates an HTTP server and supplies request and response objects to the callback.
res.end()	Ends the HTTP response and sends the specified text to the client.
server.listen(3000	Starts the HTTP server and makes it listen on port 3000.
new URL()	Creates a URL object from the supplied URL string.
myURL.protocol / host / pathname / search	Reads individual components of the URL.
EventEmitter	Provides a mechanism for registering and emitting events.
myEvent.on()	Registers a callback for the welcome event.
myEvent.emit()	Triggers the welcome event and executes its listener.

require('./app')	Imports a local module named app so its exported functions can be used.
math.add/sub/mul/div	Calls the arithmetic functions with the values 10 and 5.
express()	Creates an Express application instance.
app.get('/', ...)	Defines a GET route for the root URL.
app.listen(3000, ...).	Starts the Express application on port 3000.
const fs = require('fs')	Imports the built-in file-system module.
fs.writeFileSync()	Writes 'hello node.js' into hello.txt synchronously.
const os = require('os')	Imports the operating-system module.
os.platform()	Returns the platform/operating system identifier.
os.cpus().length	Returns the number of CPU cores reported by Node.js.
path.basename()	Returns the last portion of the specified path, which is the file name.
http.createServer()	Creates an HTTP server and supplies request and response objects to the callback.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Execute builtin.js	hello.txt should be created and 'File created' should be displayed	File creation operation executed	Pass
TC-02	Run os module statements	Platform and CPU core count should be displayed	OS details displayed	Pass
TC-03	Run path.basename()	The file name from the specified path should be displayed	File name extracted	Pass
TC-04	Run HTTP server and open port 3000	Server should respond with 'Hello from Node.js!'	HTTP response served	Pass
TC-05	Run URL parsing statements	Protocol, host, path and query should be displayed	URL components displayed	Pass
TC-06	Emit welcome event	Welcome message should be displayed	Welcome event handled	Pass
TC-07	Execute user.js	Add, Sub, Mul and Div results should be displayed	Arithmetic results displayed	Pass
TC-08	Run Express application and visit /	Express should respond with 'Hello from Express!'	Express response served	Pass

9. OUTPUT

Output 1: Node.js and npm versions verified successfully.

```
Problems  Output  Debug Console  Terminal  Ports
● PS C:\Users\FARMA\OneDrive\Desktop\almas> node --version
v22.16.0
● PS C:\Users\FARMA\OneDrive\Desktop\almas> npm --version
10.9.2
```

OUTPUT 2: Built-in Node.js modules executed successfully and their outputs were displayed.

```
Problems  Output  Debug Console  Terminal  Ports
❖ PS C:\Users\FARMA\OneDrive\Desktop\almas> node builtin.js
File created
win32
12
hello.txt
Server running on port 3000
Protocol: https:
Host: example.com:8080
Path: /products
Query: ?id=101
Welcome to Node.js!
```

OUTPUT 3: HTTP server successfully created and accessed through localhost:3000

```
<  >  ↻  ⓘ  localhost:3000

Hello from Node.js!
```

OUTPUT 4: Custom Node.js module successfully executed with arithmetic operations

```
node.js v22.16.0
● PS C:\Users\FARMA\OneDrive\Desktop\almas> node user.js
Add: 15
Sub: 5
Mul 50
Div 2
❖ PS C:\Users\FARMA\OneDrive\Desktop\almas>
```

10. RESULT

Thus, the Node.js programs were successfully implemented using built-in modules, a local custom module and Express. File creation, operating-system information, path handling, HTTP server creation, URL parsing, event handling, arithmetic module functions and an Express GET route were demonstrated according to the supplied programs.
